

کلی فاضل نیا
مدرس دوره دانشجویی: ۰۱۱۸۳۳۲۲۰

۱.۴. الگوریتم جستجوی دودویی (binary search) در آرایه مرتب شده را نوشته و پیچیدگی آن را در سه حالت بهترین، متوسط و بدترین محاسبه کنید.

1) `int binarySearch (int S[n], int L, int r, int n)`

2) `while (L <= r) {`

3) `int m = L + (r - 1) / 2;`

4) `if (S[m] == n)`

5) `return m;`

6) `if (S[m] < n)`

7) `L = m + 1;`

8) `if (S[m] > n)`

9) `r = m - 1;`

10) `return -1`

نکات مهم:

نمای

`int S[n]` آرایه مرتب شده

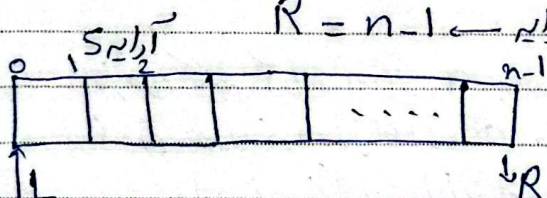
`int n = sizeof(S) / sizeof(S[0])`

L: کران یا محدوده پایین ما

R: کران یا محدوده پایین ما $R = n - 1$ طول آرایه

n: عددی که می خواهیم در آرایه پیدا کنیم

L در ابتدا صفر است



بهترین حالت وقتی است که n ما همان میانگین اول باشد و در این صورت حلقه تنها یک بار

اجرا می شود و m را و return می کند پس $T(n) \in \Omega(1)$

دستور

زمان اجرا

تعداد دفعات

بدترین حالت: بهترین حالت زمانی است که $k+1$

داخل است باشد و حلقه تا زمانی که طول آرایه به یک

برسد تکرار شود و در هر بار اجرای حلقه تعداد اعضای آرایه

به دو تقسیم می شوند لذا:

$\frac{n}{2} \rightarrow \frac{n}{4} \rightarrow \frac{n}{8} \rightarrow \frac{n}{16} \rightarrow \dots \rightarrow \frac{n}{2^k} = 1$

$\rightarrow n = 2^k \rightarrow \log n = k$

Arman

C1
C2
C3
C4
C5
C6
C7
C8
C9
C10

1

نتیجه در نتیجه $T(n) \in O(\log n)$ 1. در بدترین حالت پیچیدگی زمانی $O(\log n)$ است

page: ()

حالت متوسط (میان دگین):

$$\sum_{i=1}^n i = 2+2$$

$$3+3+3+3 = 3 \times 4$$

$$\frac{n \times 2^{n-1}}{2} \log_2 n = \frac{1}{2} \log n \Rightarrow T(n) \in \Theta(\log n)$$

۱-۵. عناصر زیر را با استفاده از الگوریتم مرتب سازی درج مرتب کنید.

$n=1=4-1=4$
 6 5 3 1 2
 $k=1 \leq 4$ $key=6$ $i=0$ $a[i]=6$ $i'=0, 6 > 5$ $a[i]=5$ $i'=-1$ $a[0]=6$
 $k=2 \leq 4$ $key=5$ $i=1$ $a[i]=5$ $i'=0, 5 > 3$ $a[i]=3$ $i'=0$ $a[0]=5$
 $k=3 \leq 4$ $key=3$ $i=2$ $a[i]=3$ $i'=0, 3 > 1$ $a[i]=1$ $i'=0$ $a[0]=3$
 $k=4 \leq 4$ $key=1$ $i=3$ $a[i]=1$ $i'=0, 1 < 2$ $a[i]=2$ $i'=0$ $a[0]=1$
 1 2 3 5 6
 $k=5 \leq 4$ $key=2$ $i=4$ $a[i]=2$ $i'=0, 2 < 6$ $a[i]=6$ $i'=0$ $a[0]=2$
 2 1 3 5 6
 $k=6 \leq 4$ $key=3$ $i=5$ $a[i]=3$ $i'=0, 3 < 5$ $a[i]=5$ $i'=0$ $a[0]=3$
 3 1 2 5 6
 $k=7 \leq 4$ $key=5$ $i=6$ $a[i]=5$ $i'=0, 5 < 6$ $a[i]=6$ $i'=0$ $a[0]=5$
 5 1 2 3 6
 $k=8 \leq 4$ $key=6$ $i=7$ $a[i]=6$ $i'=0, 6 < 5$ $a[i]=5$ $i'=0$ $a[0]=6$
 6 1 2 3 5

۱-۷. فرض کنیم آرایه n تایی A به صورت صعودی و یک n تایی هدف پیدا کردن دو اندیس i و j به طوری که $A[i] + A[j] = n$ باشد. الگوریتمی با پیچیدگی $O(n)$ ارائه کنید. عناصر، برای عناصر زیر الگوریتم ارائه شده را trace ($n=20$) کنید.
 2, 4, 7, 9, 11, 12, 17

کد پایتون	الگوریتم:
<code>list1 = [2, 4, 7, 9, 11, 12, 17]</code>	1. لیست را مقایسه می کنیم
<code>x = int(input("enter x: "))</code>	2. x را از کاربر دریافت می کنیم.
<code>for i in range(len(list1)-1):</code>	3. یک حلقه طبقاً از اقامت طول لیست - 1 روی
<code> k = x - list1[i]</code>	اکسای لیست پیاده می کنیم.
<code> if k in list1:</code>	4. هر مقولیت را از x کم می کنیم و به k می بینیم.
<code> print(list1.index(k),</code>	5. شرط می داریم که k در خود لیست است یا نه.
<code> list1.index(x - k))</code>	6. اگر شرط درست باشد، ایندکس دومی
<code> break</code>	که جمع شان x شود را به دست می کنیم.
	7. Arman
	8. اگر شرط درست نباشد به حیثان ادامه می دهیم.

کد C++ و ادامه در صفحه بعد

1) int main (void) : C++ کد

2) {

3) int a; : a کد مورد نظر

4) int n[] = {2, 4, 7, 9, 11, 12, 17}; : n لیست مورد نظر

5) int size = sizeof(n) / sizeof(int) - 1;

6) std::cout << "enter number: ";

7) std::cin >> a;

8) for (int i = 0; i <= size; i++) {

9) int result = a - n[i]

10) int *result_find = std::find(&n[0], &n[size], result);

11) if (result_find != &n[size]) {

12) std::cout << result_find - &n[0] << " " << i << std::endl;

13) break

14) }

15) }

16) return 0;

17) }

نتیجه: اگر result در n باشد، result_find = &n[size] و اگر نه، result_find = &n[0].

* و * هر دو آپرس کد مورد نظر را برمی گردانند.

a = 20 و n = [2, 4, 7, 9, 12, 17] Trace

[2, 4, 7, 9, 12, 17]

a = 20

Size = 6

مرحله اول، $i = 0 < 6$ ، $result = 20 - 2 = 18$ → 18 not in n → *result_find = [6] → result_find [17] = &n[6]

مرحله دوم، $i = 1 < 6$ ، $result = 20 - 4 = 16$ → 16 not in n → *result_find = [6] × → result_find [17] = &n[6]

مرحله سوم، $i = 2 < 6$ ، $result = 20 - 7 = 13$ → 13 not in n → *result_find = [6] × → result_find [17] = &n[6]

مرحله چهارم، $i = 3 < 6$ ، $result = 20 - 9 = 11$ → 11 in n → *result_find = [4] × → result_find [11] = &n[6]

→ cout 4, 3 → break

8 (12) - 5 (2) مرحله اول

Arman

شماره دانشجویی: ۴۰۱۱۸۲۳۲۲۰

کلی فاضل نیا

۱۵-۱ روابط بازگشتی زیر را با هر سه روش تفهیم اصلی و درخت بازگشت حل کنید.

$$a) T(n) = 2T\left(\frac{n}{2}\right) + n^2$$

$$b) T(n) = 2T\left(\frac{n}{2}\right) + n \log n$$

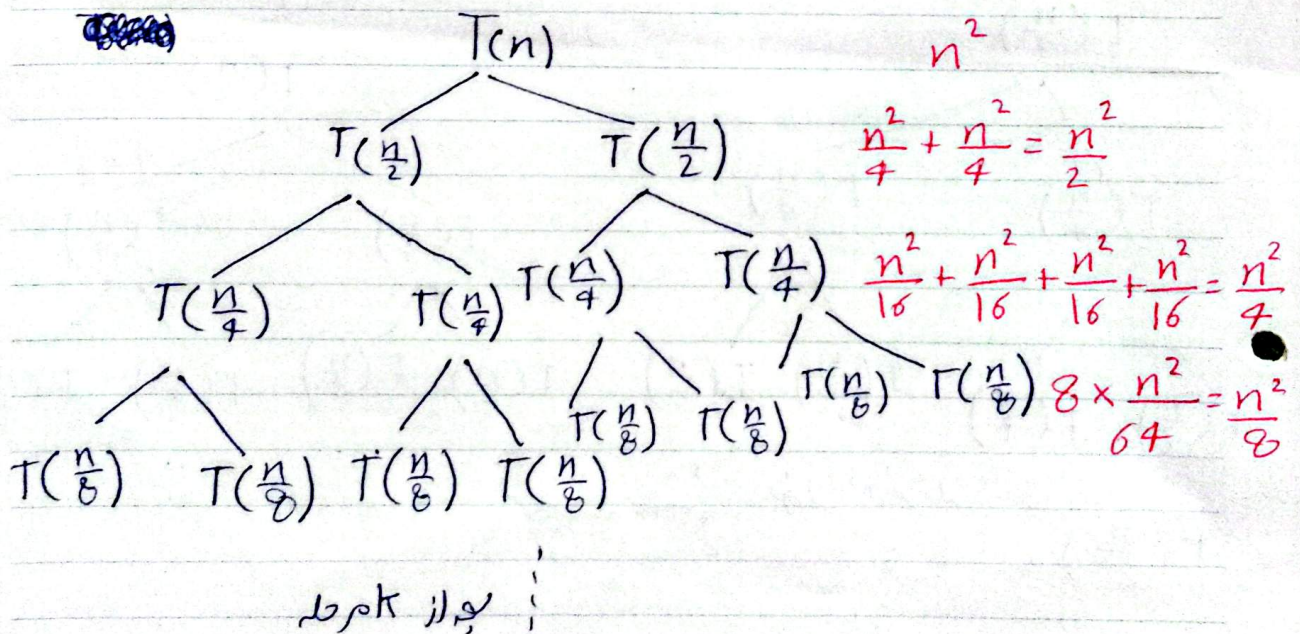
$$a) T(n) = 2T\left(\frac{n}{2}\right) + n^2$$

تفهیم اصلی: $k=0 \quad a=2 \quad b=2 \rightarrow g(n) = n^{\log_2 2} \quad G = \log_2 2 = 1$
 $f(n) = n^2 \rightarrow F = 2$

$$k=0, F > G \rightarrow T(n) \in \Theta(f(n)) \rightarrow T(n) \in \Theta(n^2)$$

$$T(n) = 2T\left(\frac{n}{2}\right) + n^2$$

درخت بازگشت:



$$T\left(\frac{n}{2^k}\right)$$

$$T(n) = n^2 + \frac{n^2}{2} + \dots = n^2 \left(1 + \frac{1}{2} + \dots\right) = 2n^2 \in \Theta(n^2)$$

$$T(n) = 2\left(2T\left(\frac{n}{2^2}\right) + \left(\frac{n}{2}\right)^2\right) + n^2 = 2^2 T\left(\frac{n}{2^2}\right) + \frac{n^2}{2} + n^2$$

تکرار و جایگذاری

Parsian

$$= 2^2 \left(2T\left(\frac{n}{2^3}\right) + \left(\frac{n}{2^2}\right)^2\right) + \frac{n^2}{2} + n^2 = 2^3 T\left(\frac{n}{2^3}\right) + \frac{n^2}{2} + \frac{n^2}{2} + n^2$$

$$\xrightarrow{\sim k} = 2^k T\left(\frac{n}{2^k}\right) + n^2 \left(\frac{1}{2^{k-1}} + \frac{1}{2^{k-2}} + \dots + \frac{1}{2^0}\right) = 2^k T\left(\frac{n}{2^k}\right) + n^2 \left(\frac{1}{1-\frac{1}{2}}\right)$$

$$\frac{n}{2^k} = 1 \rightarrow n = 2^k \rightarrow k = \log n$$

$$\Rightarrow T(n) = nT(1) + 2n^2 \in \Theta(n^2)$$

$$b) T(n) = 2T\left(\frac{n}{2}\right) + n \log n$$

$$f(n) = n \rightarrow F = 1 \quad k = 1$$

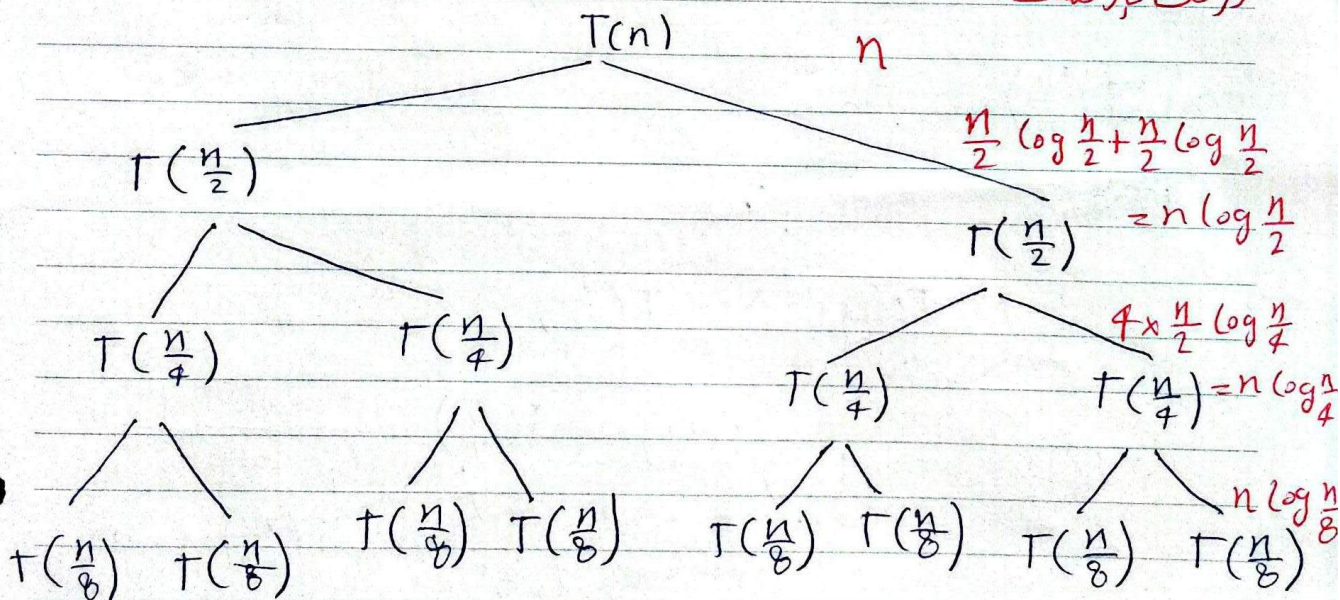
$$g(n) = n^{\log \frac{1}{2}} \rightarrow G = \log \frac{1}{2} = 1$$

قوله اولی:

$$k = 1, F = G \rightarrow T(n) \in \Theta(g(n)(\log n)^{k+1})$$

$$\rightarrow T(n) \in \Theta(n(\log n)^2)$$

درخت بازگشت:



$$\frac{n}{2^k} = 1 \rightarrow n = 2^k \rightarrow k = \log n$$

$$T(n) = n \left(\log \frac{n}{2^0} + \log \frac{n}{2^1} + \log \frac{n}{2^2} + \dots \right) = n (\log n - \log 2^0$$

$$+ \log n - \log 2^1 + \log n - \log 2^2 + \dots)$$

Parsian

$$= n (k \log n - (0+1+\dots)) \in \Theta(n(\log n)^2)$$

صافه دوم

تکرار و جابجاری

$$\begin{aligned} T(n) &= 2 \left(2T\left(\frac{n}{2^2}\right) + \frac{n}{2} \log \frac{n}{2} \right) + n \log n \\ &= 2^2 T\left(\frac{n}{2^2}\right) + n \log \frac{n}{2} + n \log n = 2^2 \left(2T\left(\frac{n}{2^3}\right) + \frac{n}{2^2} \log \frac{n}{2^2} \right) \\ &\quad + n \log \frac{n}{2} + n \log n = 2^3 T\left(\frac{n}{2^3}\right) + n \log \frac{n}{2^2} + n \log \frac{n}{2} + n \log n \end{aligned}$$

$$\begin{aligned} \text{در } k, &= 2^k T\left(\frac{n}{2^k}\right) + n \left(\log \frac{n}{2^{k-1}} + \dots + \log \frac{n}{2^0} \right) \\ &= 2^k T\left(\frac{n}{2^k}\right) + n \left(\log n - \log 2^{k-1} + \log n - \log 2^{k-2} + \dots \right) \\ &= 2^k T\left(\frac{n}{2^k}\right) + n \left(k \log n - (k-1 + k-2 + \dots + 1 + 0) \right) \\ &= 2^k T\left(\frac{n}{2^k}\right) + n \left(k \log n - \left(\frac{k(k-1)}{2} \right) \right) \end{aligned}$$

$$\frac{n}{2^k} = 1 \rightarrow n = 2^k \rightarrow k = \log n$$

$$T(n) = n T(1) + n \left((\log n)^2 - \dots \right) \in O(n (\log n)^2)$$